

Business Logic & Database Design Specification

Complete Technical Architecture for Next.js 16, Cloud Firestore, Firebase Auth & Razorpay

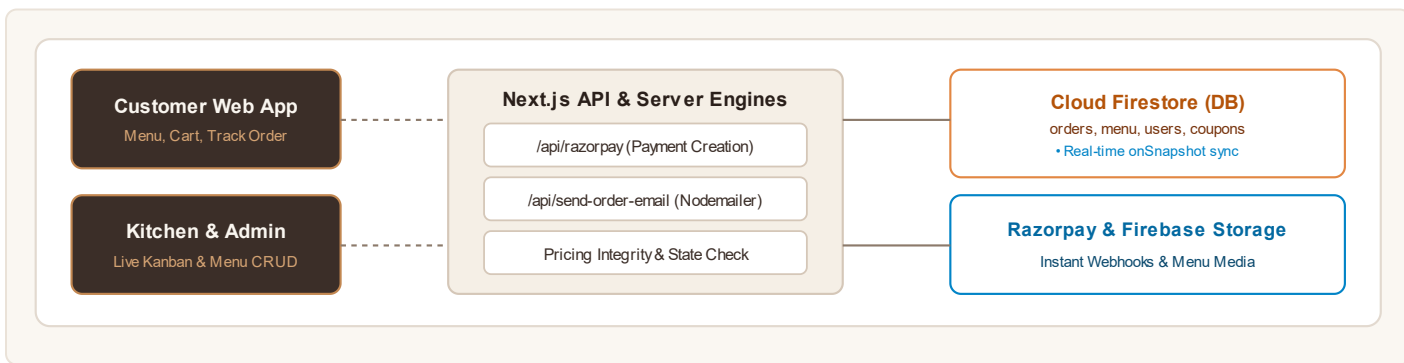
Project: 7th Heaven Cafe **Framework:** Next.js 16 (App Router) / React 19

Database: Google Cloud Firestore (NoSQL)

Payments: Razorpay **Version:** 1.0 (Production)

1 System Architecture Overview

7th Heaven Cafe is an artisanal bakehouse and specialty cafe web platform. The platform operates on a serverless, decoupled architecture utilizing **Next.js 16 App Router** for presentation and API route handlers, paired with **Firebase Cloud Firestore** for real-time document synchronization and **Razorpay** for automated payments.



2 Business Logic Architecture

2.1 Role-Based Access Control (RBAC)

ROLE	PERMISSIONS & ACCESSIBLE INTERFACES	ENFORCEMENT LAYER
Guest / Anonymous	Browse catalog, add items to client cart, guest checkout with phone/email verification, track active orders via local storage ID tokens.	Firestore Rules: Public read on <code>/menu</code> ; Read on <code>/orders</code> if <code>userId == 'guest'</code> .
Registered Customer	Save multiple delivery addresses, store profile preferences, 1-click reordering, submit post-delivery star ratings and review comments, live tracking.	Firestore Rules: Strict ownership match (<code>auth.uid == userId</code>).
Kitchen Staff	View incoming live orders in real-time on Kanban board, acknowledge receipt, update status (<code>placed</code> → <code>preparing</code>), toggle item stock.	Firestore custom user role claim <code>role: 'kitchen' or 'admin'</code> .
Delivery Courier	Inspect customer address, view delivery instructions, transition order to <code>out_for_delivery</code> and <code>delivered</code> .	Role claim <code>role: 'delivery' or 'admin'</code> .
Super Admin	Full CRUD over menu catalog, promotional discounts/coupons, real-time revenue analytics charts, customer support adjustments.	Role claim <code>role: 'admin'</code> ; full administrative Firestore access.

2.2 Order Lifecycle & State Machine

Orders strictly follow a finite state progression. Irregular state transitions (e.g. attempting to mark an order as delivered before preparing) are rejected by validation logic.



- **PLACED** : Razorpay payment verified or Cash on Delivery committed. Order enters the kitchen real-time feed.
- **PREPARING** : Kitchen staff confirms order. Estimated delivery time (EDT = `placedAt + 25 mins`) is computed and countdown activates on customer client.
- **OUT FOR DELIVERY** : Order is handed over to courier. Real-time ETA updates dynamically based on transit time.
- **DELIVERED** : Courier completes handoff. Triggers automatic delivery invoice dispatch via Nodemailer and opens feedback rating modal.
- **CANCELLED** : Permitted only within 2 minutes of placement or by Admin if item is unavailable. Triggers Razorpay refund webhook.

2.3 Pricing, Cart & Mathematical Integrity Rules

To prevent client-side price tampering, all cart items submitted to `/api/razorpay` and checkout routes are re-calculated against active database records:

Subtotal: $\text{₹} = \sum (\text{Item Unit Price} \times \text{Quantity}) + \sum \text{Selected Addons}$

Taxes (GST 5%): $\text{₹} = \text{Round}(\text{Subtotal} \times 0.05)$

Delivery Fee: ₹40 (Waived to ₹0 if Subtotal $\geq$ ₹500)

Net Payable: $\text{₹} = \text{Subtotal} + \text{Taxes} + \text{Delivery Fee} - \text{Discount}$

Historical Price Snapshot Principle: When an order document is created, the system stores a fully-frozen JSON snapshot of each ordered item (name, unit price, addons, qty, line total). Any subsequent updates to menu item prices by the cafe manager will never retroactively modify the customer's historical order value.

3 Database Design (Cloud Firestore Schema)

Cloud Firestore is designed as a document-oriented NoSQL schema optimized for high read velocity, low latency listener subscriptions, and zero duplicate writes.

3.1 Collection: users

```
// Document Path: /users/{userId}
{
  "uid": "usr_948fbc834ad",
  "name": "Aditya Sharma",
  "email": "aditya@example.com",
  "phone": "+919876543210",
  "role": "customer", // "customer" | "kitchen" | "admin"
  "photo": "https://...",
  "addresses": [
    {
      "id": "addr_1",
      "tag": "Home",
      "street": "Flat 402, Sunshine Heights, MG Road",
      "city": "Pune",
      "pincode": "411001",
      "isDefault": true
    }
  ],
  "newsletter": true,
  "createdAt": "2026-03-01T10:00:00.000Z"
}
```

3.2 Collection: menu

```
// Document Path: /menu/{menuItemId}
{
  "id": "menu_mno123",
  "name": "Caramel Flan Latte",
  "category": "Hot Coffee",
  "price": 120,
  "inStock": true,
  "isVegetarian": true,
  "isBestseller": true,
  "description": "Warm latte with flan caramel.",
  "image": "https://firebasestorage...",
  "prepTimeMinutes": 5,
  "addons": [
    { "name": "Extra Espresso Shot", "price": 30 },
    { "name": "Almond / Oat Milk", "price": 40 }
  ],
  "createdAt": "2026-01-15T08:00:00.000Z"
}
```

3.3 Collection: orders (Core Transaction Entity)

```
// Document Path: /orders/{orderId}
{
  "id": "ord_7h_8a7d2b",
  "orderNumber": "7H-20260303-0042",
  "userId": "usr_948fbc834ad",          // or "guest"
  "customerName": "Aditya Sharma",
  "customerEmail": "aditya@example.com",
  "customerPhone": "+919876543210",
  "customerAddress": "Flat 402, Sunshine Heights, MG Road, Pune - 411001",
  "orderType": "delivery",              // "delivery" | "dine_in" | "takeaway"

  // Historical Snapshot of Line Items
  "items": [
    { "menuItemId": "menu_mno123", "name": "Caramel Flan Latte", "price": 120, "qty": 2, "selectedAddons": [{ "name": "Caramel Sauce", "price": 10, "qty": 1, "selected": true }, { "name": "Mint Mojito", "price": 99, "qty": 1, "selected": false }, { "name": "Vanilla Ice Cream", "price": 20, "qty": 1, "selected": true } ], "itemTotal": 250 },
    { "menuItemId": "menu_ref456", "name": "Mint Mojito", "price": 99, "qty": 1, "selectedAddons": [], "itemTotal": 99 }
  ],

  // Financial Breakdown
  "subtotal": 399,
  "taxes": 20,                          // 5% GST
  "deliveryFee": 40,
  "discount": 0,
  "total": 459,

  // Payment Tracking
  "paymentMethod": "Online (Razorpay)", // "Online (Razorpay)" | "Cash on Delivery"
  "paymentStatus": "paid",              // "pending" | "paid" | "failed" | "refunded"
  "paymentId": "pay_PAB91823749",
  "razorpayOrderId": "order_ODZ19385012",

  // Lifecycle Progression
  "status": "placed",                  // "placed" | "preparing" | "out_for_delivery" | "delivered" | "cancelled"
  "placedAt": "2026-03-03T12:00:16.000Z",
  "estimatedDeliveryAt": "2026-03-03T12:25:16.000Z",
  "deliveredAt": null,

  // Customer Feedback
  "rating": null,                      // 1 to 5 stars
  "feedback": null                     // text comment
}
```

3.4 Collection: coupons

```
// Document Path: /coupons/{code}
{
  "code": "HEAVEN20",
  "discountType": "percentage",
  "discountValue": 20,
  "maxDiscount": 100,
  "minOrderValue": 300,
  "validUntil": "2026-12-31T23:59:59.000Z",
  "isActive": true
}
```

3.5 Collection: analytics_daily

```
// Document Path: /analytics_daily/{YYYY-MM-DD}
{
  "date": "2026-03-03",
  "totalOrders": 38,
  "totalRevenue": 14250,
  "avgOrderValue": 375,
  "topSellingItem": "Caramel Flan Latte"
}
```

4 Firestore Indexing & Query Optimization

COLLECTION	INDEXED FIELDS & DIRECTION	APPLICATION QUERY PATTERN
orders	userId (ASC) + placedAt (DESC)	Customer past order history in ProfileDrawer.
orders	status (ASC) + placedAt (DESC)	Admin Kanban view filtering by active statuses.
orders	placedAt (DESC)	Admin live activity stream and daily revenue chart.
menu	category (ASC) + price (ASC)	Category filtering & price-sorted catalog browsing.

5 Production Firestore Security Rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    function isAuthenticated() { return request.auth != null; }
    function isOwner(userId) { return isAuthenticated() && request.auth.uid == userId; }
    function isAdmin() {
      return isAuthenticated() &&
        (request.auth.token.role == 'admin' || get(/databases/{database}/documents/users/{request.auth.uid}).data.role == 'admin');
    }

    // 1. Users: Read/Write own doc; Admin full access
    match /users/{userId} {
      allow read: if isOwner(userId) || isAdmin();
      allow create: if isAuthenticated() && request.auth.uid == userId;
      allow update: if isOwner(userId) || isAdmin();
      allow delete: if isAdmin();
    }

    // 2. Menu: Public readable, Admin manageable
    match /menu/{itemId} {
      allow read: if true;
      allow write: if isAdmin();
    }

    // 3. Orders: Multi-tenant safety
    match /orders/{orderId} {
      allow read: if resource.data.userId == 'guest' || (isAuthenticated() && resource.data.userId == request.auth.uid);
      allow create: if request.resource.data.status == 'placed';
      allow update: if (isAuthenticated() && resource.data.userId == request.auth.uid &&
        request.resource.data.diff(resource.data).affectedKeys().hasOnly(['rating', 'feedback'])) ||
        isAdmin();
      allow delete: if isAdmin();
    }

    // 4. Coupons & Analytics
    match /coupons/{couponId} {
      allow read: if true;
      allow write: if isAdmin();
    }
    match /analytics_daily/{dateId} {
      allow read, write: if isAdmin();
    }
  }
}
```